

Experiment-04: Crypt-Arithmetic Problem using Python

Aim

To develop a Python program that solves a **Crypt-Arithmetic Problem** using **Backtracking**, where each letter represents a unique digit (0–9), and the arithmetic equation is satisfied.

Problem Statement

Solve the crypt-arithmetic puzzle:

SEND

+ MORE

MONEY

Rules:

- Each letter represents a unique digit.
 - No two letters have the same digit.
 - The first letter of a word cannot be **0**.
-

Algorithm

1. Identify all unique letters in the puzzle.
 2. Generate all possible digit assignments (0–9).
 3. Ensure each letter has a unique digit.
 4. Do not assign **0** to leading letters (S and M).
 5. Replace letters with their assigned digits.
 6. Check whether:
 - **SEND + MORE = MONEY**
 7. If the equation is true, display the solution.
 8. Otherwise, continue searching until a valid assignment is found.
-

Flowchart

START



Read Crypt-Arithmetic Puzzle



Identify Unique Letters



Generate Possible Digit Assignments



Are Digits Unique and

Leading Digits $\neq 0$?



No



Yes



Try Next Assignment



Does $\text{SEND} + \text{MORE} = \text{MONEY}$?



No



Yes



Try Next Assignment



Display Solution

|



STOP

Python Code

```
from itertools import permutations
```

```
letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')
```

```
digits = range(10)
```

```
for perm in permutations(digits, len(letters)):
```

```
    S, E, N, D, M, O, R, Y = perm
```

```
    # Leading digits cannot be zero
```

```
    if S == 0 or M == 0:
```

```
        continue
```

```
    SEND = 1000*S + 100*E + 10*N + D
```

```
    MORE = 1000*M + 100*O + 10*R + E
```

```
    MONEY = 10000*M + 1000*O + 100*N + 10*E + Y
```

```
    if SEND + MORE == MONEY:
```

```
        print("Solution Found\n")
```

```
        print("SEND =", SEND)
```

```
        print("MORE =", MORE)
```

```
        print("MONEY =", MONEY)
```

```
print("\nLetter Assignments")

print("S =", S)

print("E =", E)

print("N =", N)

print("D =", D)

print("M =", M)

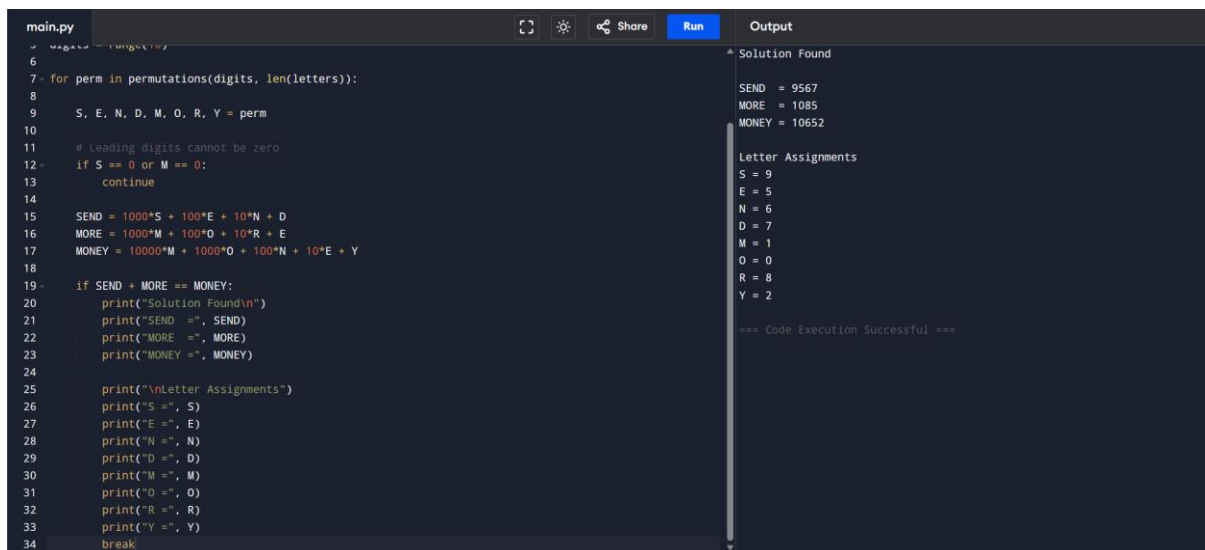
print("O =", O)

print("R =", R)

print("Y =", Y)

break
```

Output



```
main.py  [Icons]  Run  Output
6
7- for perm in permutations(digits, len(letters)):
8
9     S, E, N, D, M, O, R, Y = perm
10
11     # Leading digits cannot be zero
12-    if S == 0 or M == 0:
13        continue
14
15     SEND = 1000*S + 100*E + 10*N + D
16     MORE = 1000*M + 100*O + 10*R + E
17     MONEY = 10000*M + 1000*D + 100*N + 10*E + Y
18
19-    if SEND + MORE == MONEY:
20        print("Solution Found")
21        print("SEND =", SEND)
22        print("MORE =", MORE)
23        print("MONEY =", MONEY)
24
25        print("\nLetter Assignments")
26        print("S =", S)
27        print("E =", E)
28        print("N =", N)
29        print("D =", D)
30        print("M =", M)
31        print("O =", O)
32        print("R =", R)
33        print("Y =", Y)
34        break
```

Solution Found

SEND = 9567
MORE = 1085
MONEY = 10652

Letter Assignments

S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2

=== Code Execution Successful ===

Result

The **Crypt-Arithmetic Problem** was successfully solved using the **Backtracking (Permutation Search)** technique. The program assigned unique digits to each letter while satisfying all constraints, producing the valid solution:

- **SEND = 9567**
- **MORE = 1085**
- **MONEY = 10652**

